

Proyecto Final Integrador 2026

Descripción general

El proyecto final consiste en desarrollar una aplicación web con Angular 21 y Bootstrap 5 que simule el sitio de una empresa, emprendimiento o marca. La aplicación debe tener una estructura de navegación completa con al menos cuatro secciones, un componente reutilizable y comunicación entre componentes padre e hijo.

El tema es libre: puede ser una tienda, un portfolio, una banda de música, un restaurante, una organización, etc. Lo importante es que el código funcione, esté bien organizado y puedas explicarlo.

El tema es tuyo — elegilo con ganas

Algunos ejemplos: tienda de ropa, restaurante, banda de música, veterinaria, librería, club deportivo, portfolio personal, ONG...

Lo que no puede variar es la estructura técnica: carpetas, componentes, rutas y requisitos.

Estructura de carpetas

Tu proyecto debe respetar la siguiente organización. Generá los componentes con `ng generate component (ng g c)` para que Angular cree los archivos correctamente.

Carpeta / Archivo	Qué va ahí
<code>src/app/models/</code>	Interfaces TypeScript (producto.ts, etc.)
<code>src/app/components/navbar/</code>	Componente Navbar (ng g c components/navbar)
<code>src/app/components/footer/</code>	Componente Footer (ng g c components/footer)
<code>src/app/components/producto-card/</code>	Componente reutilizable con <code>@Input</code> / <code>@Output</code>
<code>src/app/pages/inicio/</code>	Página principal
<code>src/app/pages/productos/</code>	Listado de productos
<code>src/app/pages/nosotros/</code>	Página Nosotros
<code>src/app/pages/contacto/</code>	Formulario de contacto
<code>src/app/pages/not-found/</code>	Página de error 404
<code>src/app/app.ts</code>	Componente raíz — importa Navbar y Footer
<code>src/app/app.html</code>	Template raíz — <code><app-navbar></code> , <code><router-outlet></code> , <code><app-footer></code>
<code>src/app/app.routes.ts</code>	Definición de todas las rutas

Cómo queda app.html

El archivo app.html es el esqueleto de toda la aplicación. El navbar y el footer siempre están visibles; el `<router-outlet>` es donde Angular muestra cada página según la ruta activa.

src/app/app.html

```
<div class="d-flex flex-column min-vh-100">
  <app-navbar></app-navbar>
  <main class="flex-grow-1">
    <router-outlet></router-outlet>
  </main>
  <app-footer></app-footer>
</div>
```

Este patrón flex column + min-vh-100 en el div exterior, con flex-grow-1 en el main, hace que el footer siempre quede pegado al final aunque la página tenga poco contenido.



Rutas requeridas

Configurá todas las rutas en app.routes.ts. El navbar debe tener links con routerLink apuntando a cada ruta (sin recargar la página).

Path	Componente	Descripción
/inicio	Inicio	Página principal con presentación de la marca
/productos	Productos	Listado de productos/servicios con tarjetas
/nosotros	Nosotros	Historia, misión y visión del emprendimiento
/contacto	Contacto	Formulario de contacto con nombre y mensaje
**	NotFound	Página 404 para rutas inexistentes



Componente reutilizable

Debés crear al menos un componente reutilizable (ej: producto-card, servicio-card, artista-card) que reciba datos del componente padre mediante `@Input()` y emita algún evento al padre mediante `@Output()`.

Ejemplo — modelo (models/producto.ts)

```
export interface Producto {
  id: number;
  nombre: string;
  precio: number;
  descripcion: string;
  imagen: string;
}
```

Ejemplo — componente hijo (producto-card.ts)

```
@Input() producto!: Producto;
@Output() agregarAlCarrito = new EventEmitter<Producto>();

onAgregar() {
  this.agregarAlCarrito.emit(this.producto);
}
```

Ejemplo — componente padre (productos.html)

```
@for (p of productos; track p.id) {
  <app-producto-card
    [producto]="p"
    (agregarAlCarrito)="onAgregar($event)">
  </app-producto-card>
}
```



Control flow en la página de Productos

La página de productos debe mostrar su contenido usando el control flow moderno de Angular (@for y @if). Los productos tienen que estar definidos como un array de objetos tipados con la interface que creaste.

Separación: interface en models/ y array en la página

```
// models/producto.ts — solo la interface, nada más
export interface Producto {
  id: number;
  nombre: string;
  precio: number;
  descripcion: string;
  imagen: string;
}

// pages/productos/productos.ts — importar la interface e inicializar el array
import { Producto } from '../models/producto';
productos: Producto[] = [
  { id: 1, nombre: 'Remera', precio: 3500, descripcion: 'Talle M', imagen: 'remera.jpg' },
  { id: 2, nombre: 'Campera', precio: 12000, descripcion: 'Abrigo invierno', imagen: 'campera.jpg' },
  // ... completar con los productos del tema elegido
];
```

Template con @for y @if (productos.html)

Usá @for para recorrer el array y mostrar una tarjeta por producto. Usá @if para mostrar un mensaje alternativo cuando el array está vacío:

```
@if (productos.length > 0) {  
  <div class="row">  
    @for (p of productos; track p.id) {  
      <div class="col-md-4 mb-4">  
        <app-producto-card  
          [producto]="p"  
          (agregarAlCarrito)="onAgregar($event)">  
        </app-producto-card>  
      </div>  
    }  
  </div>  
} @else {  
  <p class="text-muted">No hay productos disponibles por el momento.</p>  
}
```

Por que track p.id?

@for requiere un track para identificar cada elemento de forma única. Siempre usa una propiedad que no se repita, como el id. Sin el track, Angular no puede actualizar la lista eficientemente.

Requisitos del proyecto

El proyecto será evaluado contra los siguientes requisitos. Todos son obligatorios:

#	Requisito	Dónde aplicarlo
1	Navbar como componente en components/navbar/	Visible en todas las rutas vía app.html
2	Footer como componente en components/footer/	Visible en todas las rutas vía app.html
3	Mínimo 4 rutas: Inicio, Productos, Nosotros, Contacto	app.routes.ts + RouterLink en navbar
4	Ruta 404 con path '**'	app.routes.ts → NotFound
5	Componente reutilizable con @Input()	producto-card recibe un Producto como @Input
6	Al menos un @Output()	Ej: emitir evento al agregar al carrito / favorito
7	Interface TypeScript para el modelo de datos	models/producto.ts (u otro modelo elegido)
8	@for para renderizar el array de productos	productos.html — una tarjeta por cada objeto del array

9	@if / @else para mensaje cuando el array está vacío	productos.html — bloque @else con texto informativo
10	Bootstrap 5 en navbar, footer y cards de productos	Clases nav, navbar-brand, card, mt-auto, etc.
11	README en GitHub con nombre del proyecto y descripción	Archivo README.md en la raíz del repositorio

Entrega

La entrega consiste en dos cosas:

1. Repositorio en GitHub con el código completo del proyecto y un README.md con el nombre del proyecto, una descripción breve y tu nombre.
2. Defensa oral en clase: vas a explicar el código ante el docente durante aproximadamente 10 minutos. No es un examen de memoria — es una conversación sobre lo que hiciste y por qué.

Importante sobre la defensa

El docente puede pedir que expliques cualquier parte del código: cómo funciona el @Input, qué hace el RouterLink, por qué usaste esa estructura de carpetas, etc. El objetivo es verificar que entendés lo que escribiste.

Un proyecto que funciona pero no podés explicar no aprueba.

Consejo final

Empezá por lo que ya sabés: creá el proyecto, armá la estructura de carpetas, generá los componentes y configurá las rutas. Una vez que la navegación funciona, construí el componente reutilizable. Dejá el README y el deploy para el final.

Ante cualquier duda, consultá angular.dev — es la documentación oficial y está en español.

Prof. Exequiel Pavon · Prof. Nicolás Chandia · CET N° 30 · Web 2 · 2026